

NeovimにおけるLua設定入門

Neovim 0.5以降では設定ファイルをLuaで記述できるようになり、Vimscriptに比べて動作が高速で柔軟性が高いことからLuaによる設定が推奨されています。Neovimは内部にLuaエンジン（LuaJIT 5.1）を組み込んでおり、プラグイン開発もLuaで活発に行われています。本稿では**NeovimユーザーがLuaを使った設定・拡張ができるようになる**ことを目的に、基本から順を追って解説します。各セクションでコード例とその実行結果を示しながら、Luaによる設定方法を学んでいきましょう。

1. Luaの基本（Neovimに特化した導入）

Luaとは何か？ Luaは軽量で組み込み用途に適したスクリプト言語で、PythonやRubyに比べて高速に動作するのが特徴です。NeovimにはこのLua（正確にはLuaJIT）のエンジンが内蔵されており、NeovimをVimscript以外の**Luaスクリプトで設定・拡張**できます。Lua自体の文法はシンプルで、C言語のような文法に似た部分もあります。ここでは詳細なLua言語解説は省き、**Neovimの設定に必要な最低限のLua知識**に絞って説明します。

- **コメント文:** Luaでは `--` から行末までがコメント、`--[[ ... ]]` で複数行コメントになります。
- **変数とスコープ:** 変数は `local` を付けるとローカル変数、それ以外はグローバル変数になります。Neovim設定では基本的にローカル変数を使います（必要に応じて `vim.g` を使ってNeovimのグローバル変数を設定）。
- **主な型:** Luaには `nil`（未定義）、`boolean`（真偽値）、`number`（数値）、`string`（文字列）、`table`（連想配列）、`function`（関数）などの型があります。特に `table` はLua唯一の複合データ構造で、配列や辞書のように使えます。
- **文字列:** シングルクォート `'...'` またはダブルクォート `"..."` で囲み、文字列連結には `..` 演算子を使います。
- **簡単な例:** 以下はLuaの簡単な例です。 `--` 以降はコメントです。

```
-- 数値や文字列の操作例
local x = 10 + 5    -- 数値の計算 (15)
local msg = "Hello " .. "Lua" -- 文字列連結 ("Hello Lua")
print(msg .. "!", x * 2)
-- 実行結果: Hello Lua! 30
```

上記の例では、`local` によりローカル変数 `x` を定義し、`..` で文字列を連結しています。`print` 関数はLuaの標準出力を表示します。このようにLuaの基本文法はシンプルで、`if` 文やループなどの制御構文もC言語風の記法です。（詳しくはLua言語の入門資料を参照してください。）

2. NeovimでLuaコードを実行する方法

Neovim上でLuaコードを実行する方法はいくつかあります。**対話的に試す方法から設定ファイルへの記述まで、順に見ていきましょう。**

- **Exコマンド `:lua`:** Neovimではコマンドラインから直接Luaコードを実行できます。例えば簡単な加算を試すには以下のように入力します。

```
:lua a = 1
:lua b = 2
:lua print(a + b)
"3
```

上記のように、`:lua` コマンドでLuaコードを1行ずつ実行できます。最後のコマンドで `a+b` の結果が表示され、`3` と出力されているのが確認できます。

- ※**ワンプoint**: `:lua` コマンドで実行するコードはその都度別コンテキストで評価されます。同じ `local` 変数にはアクセスできないため、複数行にまたがる処理をする場合は後述するスクリプトファイルや `vim.cmd` を使いましょう。

- **Luaスクリプトファイルの実行**: 複数行のLuaコードを一度に試したい場合、ファイルに書いて実行できます。例えばバッファに `test.lua` というファイルを作成し、Normalモードで `:luafile %` と実行すると**現在開いているLuaファイルを実行**できます（`%` はカレントファイルを指す特殊表現）。ショートカットとして、次のように**ファンクションキーにマッピング**しておくと便利です。

```
:nnoremap <F5> <cmd>luafile %<CR>
```

これでLuaコードを書いたファイルを開いた状態で `F5` キーを押すと、そのLuaファイルが実行されます。試行錯誤しながら設定を作るのに役立ちます。

- **:source コマンド**: 実はVimscriptの `:source` でもLuaファイルを読み込めます。`.vim` ファイルと同様に `.lua` ファイルのパスを指定して `:source` すればLuaスクリプトが実行されます。
- **Neovimの設定ファイル (init.lua)**: NeovimをLuaで設定する場合、従来の `init.vim` の代わりに `init.lua` ファイルを使用します。配置場所は `init.vim` と同じく「Neovimのコンフィグディレクトリ（通常は `~/.config/nvim/`）」です。**重要**: `init.vim` と `init.lua` は同時に存在させることはできません（どちらか一方のみが読み込まれます）。Neovim起動時に `init.lua` が実行され、そこで書いたLuaコードによって設定が適用されます。
- 初めてLua設定を使う場合は、既存の `init.vim` を `init.lua` に書き換えてみましょう（`init.vim` の内容をLuaに置き換える方法は後述します）。
- **補足**: `init.lua` の中でVimscriptを使うことも可能です。例えば `vim.cmd('colorscheme desert')` のように `vim.cmd` 関数で通常のVimscriptコマンドを実行できます。既存のVimscript知識を活かしつつ移行できるので安心です。
- **Luaモジュールの読み込み (require())**: 設定が大きくなってきた場合、Luaコードを**モジュール**として分割し、必要に応じて読み込むことができます。Neovimでは `runtimepath` 上の `lua/` ディレクトリに配置したLuaファイルを `require()` 関数でロードできます。詳しい使い方は後述「**Luaモジュール化と再利用**」の章で解説しますが、例えば `~/.config/nvim/lua/myconfig.lua` というファイルを作成し、`init.lua` 内で `require('myconfig')` と書けばそのファイルを読み込んで実行できます。`require()` は一度読み込んだモジュールをキャッシュする点（2回目以降は再実行しない）に注意が必要ですが、開発時にはキャッシュをクリアする方法もあります。これについても後ほど触れます。

以上がLuaコードをNeovim上で動かす主な方法です。次章からは具体的に**LuaでNeovim設定を記述する方法**を、基本設定から順に見ていきましょう。

3. 基本的な設定（オプション設定とグローバル変数）

まずはエディタの基本オプションを設定してみましょう。Vimscriptでは`:set`コマンドや`set`オプションを直接書いていましたが、LuaではNeovimのAPIを使って設定を行います。

オプションを設定する: `vim.opt` と `vim.o` の違い

NeovimのLua APIには2通りのオプション設定方法があります:

- **`vim.opt`**: もっとも簡単な方法。`vim.opt.xxx` に値を代入すると、`:set` コマンドと同様にオプションを設定できます。例えば `vim.opt.number = true` は `:set number` (行番号の表示) に相当します。また、`vim.opt_local` や `vim.opt_global` を使えば、ローカルオプションやグローバルオプションの設定も可能です。基本的には `vim.opt` を使えば困ることは少なく、**初心者はまず `vim.opt` を使うのが無難**です。
- **`vim.o`** 等: もう一つは**変数のようにオプションにアクセスする方法**です。`vim.o` はグローバルオプション、`vim.wo` はウィンドウローカルオプション、`vim.bo` はバッファローカルオプションに対応し、それぞれ `:set`、`:setlocal`、`:setlocal` (バッファ版) のように振る舞います。例えば `vim.o.number = true` とすると行番号表示を有効にします。`vim.bo.shiftwidth = 4` とすれば現在のバッファのシフト幅のみ4に設定できます。`vim.o` 系列は**値を直接取得できる**利点があります (`print(vim.o.number)` で現在値を表示可能) が、`vim.opt` と比べると若干挙動が直感的でない場合もあります。基本的には `vim.opt` で十分ですが、**オプション値をそのまま取得したい場合や細かく制御したい場合に `vim.o` や `vim.bo` 等を使うとよいでしょう。**

では具体例を示します。Lua設定でよく使う基本オプションをいくつか設定してみます。

-- Luaによる基本的なオプション設定例

```
vim.opt.number = true      -- 行番号を表示 (:set number)
vim.opt.relativenumber = true -- 相対行番号を表示 (:set relativenumber)
vim.opt.wrap = false       -- 長い行で折り返さない (:set nowrap)
vim.opt.mouse = 'a'        -- マウス操作を有効化 (:set mouse=a)
vim.opt.clipboard:append({'unnamedplus'}) -- クリップボードを共有 (:set clipboard+=unnamedplus)

print(vim.opt.wrap:get(), vim.o.mouse)
-- 実行結果: false a
```

上の例では、`vim.opt` で代表的な設定項目を変更しています。

`vim.opt.clipboard:append({'unnamedplus'})` のように、**リスト型のオプションはLuaテーブルとして設定・追加**できます。例えばVimscriptで `set clipboard+=unnamedplus` と書いていた設定は、Luaではこのように `:append` メソッドで実現できます。同様に `:remove` や `:prepend` メソッドも用意されています。コード末尾の `print` では、設定を確認するために `vim.opt.wrap:get()` と `vim.o.mouse` を出力しています。`vim.opt` で取得したオプションは一度 `get()` を呼ぶ必要がありますが、`vim.o` なら直接値を取得できることが分かります。

Note: Vimscriptで `set noXXX` のように**no**を付けて無効化するオプション**は、Luaでは対応するオプションに `false` を代入します。例えば `set noswapfile` はLuaでは `vim.opt.swapfile = false` となります。また `set mouse=a` のように値を持つオプションは文

字列として扱う必要があるため、上記例のようにクォートで囲んで代入してください。

`vim.opt.noswapfile = true` のように `no` 付きの名前をそのまま使うとエラーになりますので注意しましょう。

グローバル変数の設定: `vim.g` を使う

NeovimではVimscript同様に**変数**も設定できます。LuaからVimscriptのグローバル変数 (`g:変数名`) にアクセスするには、`vim.g.変数名` を使用します。プラグインの設定項目などで、かつて `let g:xxx = ...` としていたものはLuaでは `vim.g.xxx = ...` と書けば同じ効果になります。

```
-- グローバル変数の設定例 (vim.g)
vim.g.mapleader = " " -- リーダーキーをスペースに設定 (let g:mapleader = " " に相当)
vim.g.auto_save = 1   -- (例) カスタム変数の設定
vim.g.some_global_table = { key1 = "val1", key2 = 42 }

print(vim.g.mapleader) -- 実行結果: <空白> (設定したリーダーキー)
```

上記では、`vim.g.mapleader` でリーダーキーをスペースに変更しています。`<Leader>` キーはプラグインのキーバインドで頻繁に使われるプレフィックスキーで、デフォルトは「`\`」ですが、スペースに変更するのが一般的です。Luaで `vim.g.mapleader = ' '` と設定すれば問題ありません。この設定行は**他のキーマッピングを行う前に実行**してください（そうしないと `<Leader>` を使ったマップ定義が正しく行えません）。

`vim.g` で設定した変数はVimscriptから見ると `g:` スcopeの変数になります。Luaでは他にも現在のバッファやウィンドウに紐づく変数を `vim.b` (buffer変数)、`vim.w` (window変数) のように操作できます¹。ただしこれらは高度な使い方になるので、本稿では必要に応じて触れる程度にします。基本的な設定では `vim.g` でグローバル変数を設定する場面が多いでしょう。

4. キーマッピングの追加とLua関数のバインド

エディタのキーバインド（マッピング）の設定もLuaで行うことができます。Vimscriptでは `nnoremap` や `vnoremap` コマンド等で設定していましたが、Luaでは専用のAPI関数を使います。

`vim.keymap.set` を使ったキー割り当て

Neovim 0.7以降では、Lua用に高水準のマッピング関数 `vim.keymap.set()` が用意されています。この関数は**モード・押下するキー・実行するアクション**の3つを引数に取り、オプション（必要なら4つ目の引数にテーブルで指定）を設定できます。例えばノーマルモードで `<Leader>w` に保存コマンドを割り当てるには次のように書きます。

```
-- キーマッピングの例

-- ノーマルモードで <Leader>w を押したら現在のファイルを保存
vim.keymap.set('n', '<Leader>w', '<cmd>write<CR>')

-- ノーマルモードで <Leader>h を押したらLua関数を実行
vim.keymap.set('n', '<Leader>h', function()
  print("Hello! これはLuaからのメッセージです。")
end)
```

```
-- ノーマル・ビジュアル両モードで <Leader>q を押したらファイルを閉じる
vim.keymap.set({'n','v'}, '<Leader>q', '<cmd>quit<CR>', { silent = true })
```

1つ目のマッピングはノーマルモード (`'n'`) に対して `<Leader>w` を押すと `:write` (保存) コマンドを実行する設定です。2つ目はノーマルモードで `<Leader>h` を押すと匿名のLua関数が呼ばれ、メッセージを出力します。3つ目はリストでモードを指定し、ノーマルモードとビジュアルモードの両方で `<Leader>q` を押すと `:quit` が実行されるマッピングです。オプションとして `{ silent = true }` を渡しているのも、このコマンド実行時の表示 (例えば `"Press ENTER to continue"` などのプロンプト) を抑制しています。

vim.keymap.set の便利な点: 従来の `vim.api.nvim_set_keymap` と違い、デフォルトで再帰的マッピングをしない (noremap相当) 設定になっています。そのため上記のようにシンプルに書くだけで **nnoremap に近い挙動** になります。また、右辺にLua関数を直接指定できるため、複雑な処理も容易に組み込めます。必要に応じて第四引数で `noremap = false` (または `remap = true`) を指定すれば再帰的なマップにもできますし、`expr=true` や `buffer=true` など他のオプションもテーブルでまとめて指定できます。

Note: ここで使用した `<cmd>...<CR>` という表記は、LuaのマッピングでVimscriptのExコマンドを実行する一種のおまじないです。例えば `'<cmd>write<CR>'` はエンターキー押下まで含めて実行する `:write` コマンドを表します。 `vim.keymap.set` で文字列を右辺に指定するときは、上記のように `<cmd> ... <CR>` 形式で書くとお互いの見通しが良くなります (もちろん `vim.cmd()` を使っても同様のことは可能です)。

なお、古い情報では `vim.api.nvim_set_keymap('n', 'キー', 'コマンド', {opts})` を使う例もありますが、現在は `vim.keymap.set` を使う方法が主流です。ただし**機能的にはどちらも実現可能**なので、既存の設定をLuaに移植する際にはどちらを使っても構いません。初学者にはオプション指定がシンプルな `vim.keymap.set` をおすすめします。

キーマップ設定の実行例

上記で設定したマッピングのいくつかを実際に動かしてみましょう (擬似的な実行例) :

```
" <Leader>w マッピングの実行例 (ファイル保存)
:w                               " <Leader>w を押したのと同じ効果でファイルが保存される

" <Leader>h マッピングの実行例 (Lua関数の実行)
:normal <Leader>h
" Hello! これはLuaからのメッセージです。

" <Leader>q マッピングの実行例 (ファイルを閉じる)
:normal <Leader>q
" (現在のウィンドウが閉じられる)
```

上では `<Leader>w` のマップでファイル保存、`<Leader>h` でメッセージ出力、`<Leader>q` でウィンドウ閉鎖が行われることを示しています。それぞれ実際にはキーボードで `<Leader>` (スペース) + 対応するキーを押下して効果を確認してください。

5. オートコマンド（自動実行）の書き方

次に、特定のイベント発生時に自動で実行されるオートコマンド（autocmd）をLuaで設定してみます。Vimscriptでは autocmd コマンドと augroup を使っていましたが、Luaでも相当するAPIが提供されています。

vim.api.nvim_create_autocmd によるオートコマンド設定

NeovimのLua APIでは、オートコマンドグループを作成する vim.api.nvim_create_augroup() と、実際のオートコマンドを作成する vim.api.nvim_create_autocmd() 関数を使います。基本的な手順は以下の通りです。

1. **グループの作成（任意）**：オートコマンドをグループ化したい場合、vim.api.nvim_create_augroup("グループ名", { clear = true }) で新しいグループを作ります。clear=true により同名グループの既存のautocmdを事前にクリアでき、重複定義を防ぎます。グループは省略も可能ですが、設定の整理や競合防止のために使うことを推奨します。
2. **オートコマンドの登録**：vim.api.nvim_create_autocmd("イベント名", { ... }) を呼び出し、イベント（例えば BufRead や BufWritePost など）とオプションを指定します。オプションには pattern（対象パターン）や callback（Lua関数）、command（文字列コマンド）などを含むテーブルを渡します。

具体例を見てみましょう。「Markdownファイルを開いたらスペルチェックを有効にする」オートコマンドをLuaで書いてみます。

```
-- Markdownファイルを開いたらスペルチェックをONにするオートコマンド
local mygroup = vim.api.nvim_create_augroup("MyAutoCmds", { clear = true })
vim.api.nvim_create_autocmd("FileType", {
  group = mygroup,
  pattern = "markdown", -- Markdownファイルタイプに対して
  callback = function()
    vim.opt_local.spell = true
    vim.opt_local.spelllang = { 'en', 'cjk' }
    print("Markdownモード: スペルチェックを有効化しました")
  end
})
```

上のコードでは、新しいグループ MyAutoCmds を作成し、その中に FileType イベント用の autocmd を1つ登録しています。pattern = "markdown" とすることで、ファイルタイプが markdown のときにのみ発火します。callback には匿名関数を渡し、その中で vim.opt_local.spell = true のように現在のバッファローカルなオプション（spell）をオンにしています。結果として、拡張子が .md などのMarkdownファイルを開くと自動でスペルチェックが有効になり、メッセージを表示するようになります。

オートコマンドの実行例:

```
" Markdownファイルを開く
:edit example.md
" Markdownモード: スペルチェックを有効化しました
```



```
:set spell?  
" spell
```

上記はMarkdownファイルを開いた際のメッセージ出力と、実際に `spell` オプションが有効になっていることを確認した例です。 `vim.opt_local` を使うことで **現在のバッファにのみ有効なオプション**を設定できる点にも注目してください。

従来のautocmdとの比較

従来Vimscriptでは以下のように書いていたものと**ほぼ同等のこと**がLuaで可能です。

```
" Vimscriptでの従来のautocmd例（参考）  
augroup MyAutoCmds  
  autocmd! " 既存の同名グループをクリア  
  autocmd FileType markdown setlocal spell spelllang=en,cjk  
augroup END
```

Luaの場合でもグループ作成とコマンド登録の概念は同じです。違いとして、Lua版の `nvim_create_augroup` では `clear=true` を指定するだけで前の `autocmd!` が不要になっていたり、 `nvim_create_autocmd` で複数イベントをテーブルで指定できたりします。例えば `events = {"BufEnter", "BufWinEnter"}` のようにイベント名を配列で渡せば、複数イベントに対して同じ処理をまとめて設定できます。

Note: `callback` ではなく `command` キーを使って文字列でコマンドを書くこともできます。例えばLuaでAutocmdを定義しつつ中身はVimscriptコマンドを実行させることも可能です。必要に応じて使い分けてください。

6. Luaモジュール化と再利用の方法

設定が大きくなってきたら、**Luaモジュールとしてファイルを分割**し、必要に応じて読み込むことで整理すると便利です。先述したように、Neovimでは `runtimepath` 上の `lua/` ディレクトリ以下に配置したLuaファイルを `require()` 関数でロードできます。モジュール化することで設定の再利用や保守が容易になります。

ファイルを分割し `require` で読み込む

例えば、設定ファイルを以下のような構成に分割するとします:

```
~/.config/nvim/  
├── init.lua  
└── lua/  
    ├── options.lua    (基本オプション設定)  
    ├── keymaps.lua    (キーマッピング設定)  
    └── myutil.lua      (自作のユーティリティ関数)
```

`init.lua` では、各設定ファイルを `require` で呼び出すだけにします:

```
-- init.luaの例（各モジュールをロード）  
require('options')
```

```
require('keymaps')
require('myutil')
```

このように書いておけば、Neovim起動時に `options.lua` や `keymaps.lua` といったファイルが順次読み込まれ、設定が適用されます。

各Luaファイルでは、必要な設定を記述します。たとえば `options.lua` には基本的なオプション類を、`keymaps.lua` にはキーマップ設定を、それぞれ前章までに説明した方法で書きます。

モジュールとして機能を定義する

`require('ファイル名')` でLuaファイルを読み込むとき、そのファイル内で**戻り値 (return)** を指定しておくと、`require` 呼び出し側で値を受け取ることができます。これを利用して**自作モジュール**を作成し、関数や変数をエクスポートして再利用することも可能です。

例えば先ほど構成例にあった `myutil.lua` を、以下のように**モジュール定義**してみます。

```
-- lua/myutil.lua (ユーティリティモジュールの例)
local M = {} -- モジュールテーブル

-- 単純な挨拶メッセージを表示する関数
function M.say_hello()
  print("Hello from myutil!")
end

-- テーブル内の要素数を出力する関数
function M.print_table_length(tbl)
  print("Table has " .. vim.tbl_count(tbl) .. " items")
end

return M -- Mテーブルをモジュールの戻り値として返す
```

上記の `myutil.lua` モジュールでは、`M` というテーブルに関数を格納し、最後に `return M` でそれを返しています。こうすると、このファイルを `require` したときに `M` テーブルが取得できます。

実際にこのモジュールを使うには、たとえば任意のLuaコードで次のようにします。

```
local util = require('myutil') -- myutilモジュールをロード
util.say_hello()
util.print_table_length({1,2,3})
-- 実行結果:
-- Hello from myutil!
-- Table has 3 items
```

`require('myutil')` を呼ぶと `lua/myutil.lua` が読み込まれ、戻り値としてモジュールテーブルが返ります。変数 `util` にそれを保持し、`util.say_hello()` のようにして関数を利用しています。2つ目の関数 `print_table_length` では、Neovim組み込みの `vim.tbl_count` 関数（テーブル中の項目数を数える便利関

数) を使ってテーブル長を表示しています。実行結果の通り、`myutil.lua` 内で定義したメッセージ関数やロジックをどこからでも呼び出せるようになりました。

Tips:

- 上記のように分割した設定ファイルやモジュールは、`init.lua` から読み込む順序を工夫することで互いに依存する設定も可能です。例えば基本オプション設定→プラグイン設定→キーマップ設定の順で `require` するなど、順序立てて記述できます。
- `require` は一度モジュールを読み込むと結果をキャッシュします。同じモジュール名を2回目に呼んだ際は即座にキャッシュを返し、ファイルを再実行しません。開発途中で何度も読み込み直したい場合は、`package.loaded['モジュール名'] = nil` でキャッシュをクリアしてから `require` すると再実行できます。
- Neovimでは `after/plugin/` ディレクトリや `plugin/` ディレクトリに置いたLuaファイルも自動読み込みされます。プラグイン的な設定をLuaで書く場合にはこれらの場所も活用できますが、基本的な自作設定は `lua/` フォルダ + `require` による管理で十分でしょう。

7. ユーザー定義コマンドの作成

最後に、ユーザー定義コマンド（Exコマンド）をLuaで作成する方法を説明します。これは `:CommandName` のようにコマンドラインから呼び出せる独自コマンドを登録する仕組みです。

`vim.api.nvim_create_user_command` でコマンド登録

NeovimではLuaから `vim.api.nvim_create_user_command('コマンド名', 実行内容, オプション)` 関数を使ってユーザーコマンドを作れます。実行内容はLua関数または文字列（Vimscriptコマンド）で指定可能です。オプションでは引数や補完の設定などができますが、ここでは基本形のみ扱います。

例: シンプルな挨拶コマンドを作ってみます。

```
-- ユーザー定義コマンドの例
vim.api.nvim_create_user_command('Hello', function(opts)
  print("こんにちは！ Luaからのご挨拶です。")
end, {})

-- 別の例: カラースキームをトグルするコマンド
vim.api.nvim_create_user_command('ToggleBg', function()
  if vim.o.background == "dark" then
    vim.o.background = "light"
    print("背景色をlightに変更しました")
  else
    vim.o.background = "dark"
    print("背景色をdarkに変更しました")
  end
end, {})
```

上記では、`:Hello` というコマンドと `:ToggleBg` というコマンドを登録しています。`:Hello` は実行するとメッセージを表示するだけの簡単なものです。`:ToggleBg` はNeovimの配色の背景(`background` オプション)を `dark` と `light` で切り替えるコマンドです。状態を判定して `vim.o.background` を変更し、変更後の状態をメッセージで通知しています。

ユーザーコマンドの実行例:

```
:Hello
" こんにちは！Luaからのご挨拶です。

:ToggleBg
" 背景色をlightに変更しました
:ToggleBg
" 背景色をdarkに変更しました
```

上記の例では、カスタムコマンド `:Hello` を実行するとLuaの関数が呼ばれ、挨拶メッセージが表示されることがわかります。また `:ToggleBg` コマンドを2回続けて実行し、背景色の切り替えメッセージが変わっている様子を確認できます。実際にはメッセージだけでなく本当に背景のテーマがライト/ダークに切り替わっています（`background` オプションはカラースキームの配色モードに影響します）。

このように、Luaでユーザー定義コマンドを作ることによって**設定や機能をコマンド名で呼び出せる**ようになります。オプション引数を取るコマンドや範囲指定に対応するコマンドも作成できますが、それらはNeovim公式の `:h command` やLua APIドキュメントを参照してください。

参考: 従来のVimscriptではユーザーコマンドを `:command!` で定義していました。Luaでは `vim.api.nvim_create_user_command` を使うことで同様のことが可能です。たとえばVimscriptで `command! Hi echo "Hi!"` と書いていたのは、Luaでは `vim.api.nvim_create_user_command('Hi', function() print("Hi!") end, {})` と表現できます。

8. おすすめの練習課題・実例

最後に、LuaでNeovim設定を書く練習のための**ミニ課題や実例**を紹介します。以下に挙げるのは実用的かつLua設定の理解が深まる例です。是非チャレンジしてみてください。

例1: カラースキーム切り替え機能の実装

目的: コマンドやキー操作でカラースキーム（配色テーマ）を切り替えるLua関数を作る。

内容: - あらかじめ使用可能なカラースキーム名のリストをLuaテーブルで用意し、ボタンを押すごとに次のテーマに変更する関数を作成してみましょう。または、ダークテーマとライトテーマをトグルで切り替えるようなコマンドでも構いません。 - カラースキームの変更は `vim.cmd("colorscheme name")` で行えます。LuaからVimの `:colorscheme` コマンドを呼びイメージです。 - 切り替え後にメッセージを表示して、どのテーマになったか分かるようにしてみましょう。

ヒント: 例えば以下のような実装が考えられます。

```
-- カラースキーム一覧から順に切り替えるサンプル
local colorschemes = { "desert", "industry", "elflord" }
local index = 1
vim.keymap.set('n', '<Leader>c', function()
  index = index % #colorschemes + 1
  local name = colorschemes[index]
```

```
vim.cmd("colorscheme " .. name)
print("Color scheme changed to: " .. name)
end)
```

上記では、<Leader>c キーを押すたびに colorschemes リストの次のテーマを適用し、メッセージを出力します。Neovim既定のいくつかのテーマ名を例示していますが、お好みでインストールしたテーマ名に変えても良いでしょう。

例2: ファイルタイプ別の設定を行う

目的: 特定のファイルタイプに対して自動的に設定を適用する。

内容: - 前章の autocmd で示したように、FileType イベントを使ってファイルタイプごとに設定を行ってみましょう。たとえば： - Markdown ファイルを開いたら自動でスペルチェックや自動折り返しをオンにする。 - Python ファイルではタブ幅を4に設定し、保存時に自動フォーマットを実行する。 - 特定のファイルタイプでのみ有効なキーマップを設定する（例: gitcommit ファイルタイプで q キーに :quit を割り当てる など）。

ヒント: Lua では vim.opt_local を使うと現在のバッファに対する設定ができます。したがって、autocmd の callback 関数内で vim.opt_local.tabstop = 4 のように書けばそのバッファだけに適用されます。また、autocmd の pattern を工夫することで複数のファイルタイプにマッチさせることも可能です（例: pattern = {"python", "lua"} のようにテーブルで指定）。

```
-- Pythonファイルを保存すると自動で`Black`（フォーマッタ）を実行する例
vim.api.nvim_create_autocmd("BufWritePost",{
  group = vim.api.nvim_create_augroup("Fmt",{ clear = true }),
  pattern = "*.py",
  command = "Black" -- 適宜インストールしたフォーマッタのコマンド
})
```

上記はBlackというPython用フォーマッタをインストールしている場合の例ですが、ファイル保存後に自動整形する典型的な autocmd になります。自分の環境に合わせてコマンド部分を変更してください。

例3: ちょっとしたプラグイン的スクリプトを書く

目的: Lua で簡単な Neovim 拡張（プラグイン）を書いてみる。

内容: - Vimscript で書かれた小さなプラグインを Lua に書き換えるつもりで、何か機能を作ってみましょう。 - 例: バッファ内の行をシャッフルするコマンド、テキストを暗号化・復号するコマンド、クリップボードの履歴を管理する機能など。

ヒント: プラグインとして配布することも考えるなら、plugin/ ディレクトリに Lua ファイルを置けば Neovim 起動時に自動ロードされます。またコマンド定義や autocmd、キー設定を組み合わせで一連の機能を作るとプラグインらしくなります。小さなものから挑戦してみてください。

以上、Lua を用いた Neovim 設定の基礎から応用までを駆け足で紹介しました。最初は Vimscript との書き方の違いに戸惑うかもしれませんが、慣れるとスクリプト言語ならではの柔軟な設定が可能になります。公式ド

コメント (`:help lua-guide`) やコミュニティの配置している設定例も参考に、ぜひ自分好みのNeovim環境をLuaで作り上げてください。

1 lua.txt

file:///file_00000000bcac6206b1ba72a9dad43eb8